

16-745: Assignment 1

Exploring Function Optimization and Using It To Optimize a Policy

Fawaz

Contents

1	Part 1a: Function Optimization on the 4D Banana Function	2
2	Part 1b: Optimizing a Learned Version of the Banana Function	3
2.1	Setup	3
2.2	Results	3
2.3	What Goes Wrong and Right	3
2.4	Difference Between Analytic and Learned Optimization	3
2.5	Advice	3
3	Part 2a: Parametric Policy Optimization — Linear Gains	4
3.1	Approach 1: Manual Tuning	4
3.2	Approach 2: LQR	5
3.3	Approach 3: Direct Optimization with <code>fminunc</code>	6
3.4	Summary	7
4	Part 2b: Neural Network Policy	8
4.1	Architecture	8
4.2	Approach 1: Direct Optimization	8
4.3	Approach 2: Imitation Learning + Fine-Tuning	8
4.4	What Changes with a Neural Network?	10
5	What I Learned	10

1 Part 1a: Function Optimization on the 4D Banana Function

The 4-dimensional Rosenbrock banana function is:

$$f(x) = \sum_{i=1}^3 [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2] \quad (1)$$

Global minimum at $x^* = (1, 1, 1, 1)$ with $f = 0$. Local minimum near $(-1, 1, 1, 1)$ with $f \approx 4$.

Two MATLAB optimizers were applied:

- **fminsearch** — gradient-free (Nelder-Mead simplex)
- **fminunc** — gradient-based (quasi-Newton)

When analytic gradients are available, **fminunc** converges more reliably and with fewer function evaluations. **fminsearch** can stall in the flat region of the banana valley between the saddle and global minimum. The specific starting point $x_0 = [-0.1565, 0.8315, 0.5844, 0.9190]$ was noted to lead to the local minimum of $f = 4$ with some methods.

2 Part 1b: Optimizing a Learned Version of the Banana Function

2.1 Setup

A neural network (`fitnet`, 10 hidden neurons) was trained on N uniform random samples from $[-2, 2]^4$ to approximate the banana function. The learned function was then optimized with both a constrained optimizer (`fmincon`, $x \in [-2, 2]^4$) and an unconstrained optimizer (`fminsearch`).

2.2 Results

Training pts	Fit at (1, 1, 1, 1)	True = 0	Fit at (-1, 1, 1, 1)	True = 4
100	242.29		124.96	
500	114.24		44.71	
1000	-13.55		-6.46	

Training pts	Constrained x	fval	True cost (con)	True cost (unc)
100	$[-0.186, -0.420, 0.082, 0.905]$	-145.14	106.48	106.48
500	$[0.619, 0.065, -0.497, -0.075]$	-153.81	48.79	48.80
1000	$[1.063, 1.122, 1.220, 1.485]$	-20.75	0.22	3.39

2.3 What Goes Wrong and Right

What goes wrong: The network hallucinates. With 1000 training points it predicted -13.55 and -6.46 at the known minima impossible, since the banana function is a sum of squares and can never be negative. When the optimizer encounters these hallucinated regions, it follows them aggressively, finding points with $f_{\text{learned}} = -145$ or -153 that have true costs of 106 and 49. The optimizer is doing its job correctly; the model is wrong.

What goes right: More data consistently helps. With 1000 points, the constrained optimizer finds a point with true cost 0.22 very close to the global minimum. Constraints matter: at 1000 points the constrained result (0.22) is much better than unconstrained (3.39), confirming that keeping the optimizer within the training distribution reduces exploitation of extrapolation errors.

2.4 Difference Between Analytic and Learned Optimization

An analytic function has a true minimum that does not change regardless of the optimizer. A learned function has no “true” minimum only an approximation. If the network hallucinates a low-cost region, the optimizer will find it and declare victory at a wrong point. It cannot distinguish a real minimum from a model artifact. With unconstrained optimization, the hallucinated values are particularly dangerous because the optimizer can extrapolate indefinitely into regions with no training data.

2.5 Advice

Give the model more training data and constrain the optimizer to the training distribution. Unconstrained optimization of a learned function is dangerous because the optimizer will find and exploit the worst extrapolation errors.

3 Part 2a: Parametric Policy Optimization — Linear Gains

3.1 Approach 1: Manual Tuning

The first attempt was manual tuning: changing the four gains $K_c = [K_1, K_2, K_3, K_4]$ by hand and rerunning the simulation. Approximately 30–40 combinations were tried over about an hour, including $[5, 1, 50, 5]$, $[10, 1, 100, 10]$, $[20, 5, 200, 20]$, and many others. Every single time the robot fell within one second. Figure 1 shows a representative run.

Testing $K_c = [0, 0, 0, 0]$ (no control) ran the full 6 seconds because the robot starts perfectly upright body angle zero, no tilt to correct. Final state: $[24.8, 0, 0, 0]$ large wheel angle but body angle zero. With initial condition $[0, 0, 0.1, 0]$ (small body tilt), zero gains caused immediate falling.

Conclusion: Manual tuning is a waste of time for more than two parameters. You cannot guess four numbers that work for a nonlinear system.

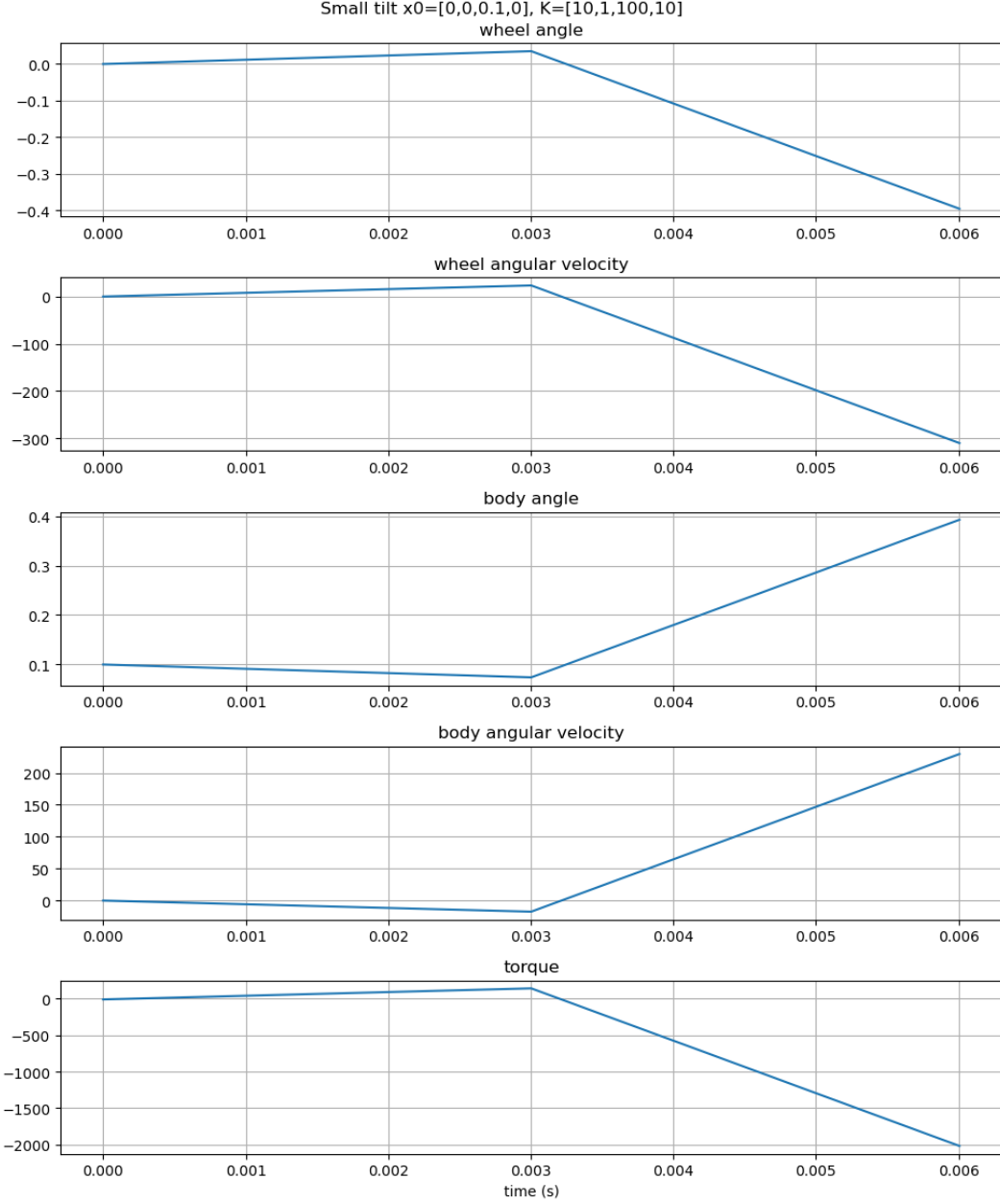


Figure 1: Simulation with $K_c = [10, 1, 100, 10]$ from $x_0 = [0, 0, 0.1, 0]$. Robot falls at step 2 (0.006 s) body angle spikes to ± 1.5 rad immediately. Gains of this magnitude cause torques large enough to destabilize the integration.

3.2 Approach 2: LQR

The TWIP dynamics were linearized around the upright equilibrium using finite differences on `twip3`, yielding the A and B matrices. MATLAB's `lqr` was called with $Q = \text{diag}([1, 1, 100, 10])$ and $R = 0.01$, yielding gains approximately $K_c \approx [316, 43, 316, 58]$.

Result: Works for small body tilt. Fails for the original initial condition (large wheel angle). LQR is a local controller it only guarantees stability near the equilibrium point where the linearization

is accurate. For large initial wheel angles, the linearization error is too large.

3.3 Approach 3: Direct Optimization with `fminunc`

A cost function was written running the full nonlinear simulation and returning negative survival time. Starting from $K_0 = [10, 1, 100, 10]$, `fminunc` ran for several hundred evaluations. The optimizer found gains that kept the robot upright for a couple of seconds but eventually fell the body angle plot appeared flat but crossed 1.5 rad.

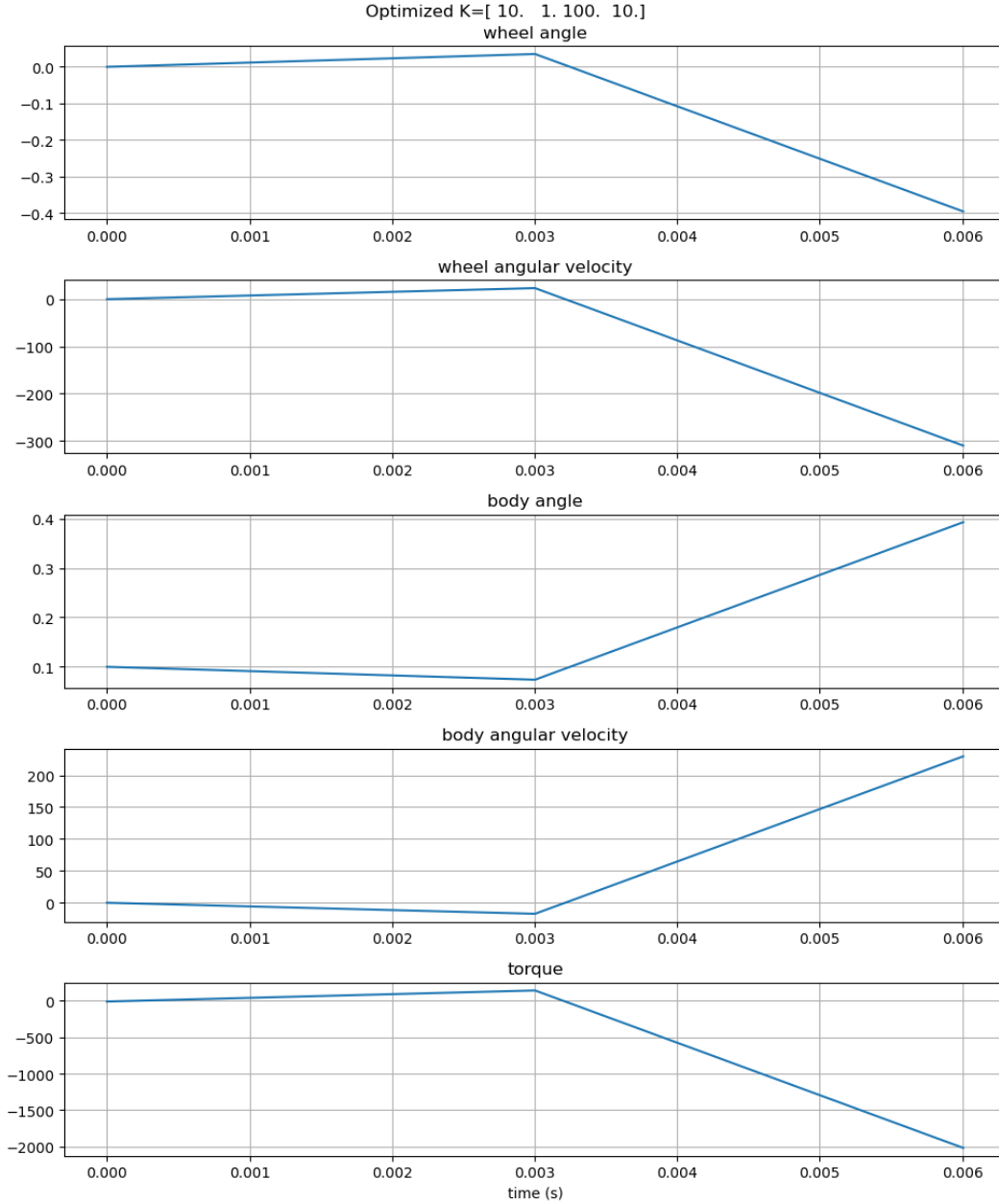


Figure 2: Simulation with optimizer-found gains. Behavior similar to the manual gains numerical instability from large velocity feedback terms dominates.

3.4 Summary

Method	Result	Why
Manual tuning	Failed	Can't guess 4 numbers for a nonlinear system
LQR	Works locally	Only valid near linearization point
fminunc	Partial	Non-smooth cost; survival-time objective tractable for 4 params

4 Part 2b: Neural Network Policy

4.1 Architecture

MATLAB's `fitnet` with one hidden layer (2 neurons, tanh activations), one linear output. Total parameters: 13 weights.

$$u = W_2^\top \tanh(W_1 x + b_1) + b_2 \quad (2)$$

4.2 Approach 1: Direct Optimization

Running `fminunc` directly on the 13-weight vector starting from random weights was a complete failure. The optimizer barely moved. 13-dimensional search space, noisy cost landscape, each evaluation requiring a full simulation. After hundreds of iterations: no improvement.

4.3 Approach 2: Imitation Learning + Fine-Tuning

The LQR gains from Part 2a were used to generate training data. 1000 random states were sampled from $[-1, 1]^4$ and target torques computed as $u = -K_{\text{LQR}}x$. The network was trained with `fitnet` to mimic this linear controller.

Post-mimic, the network successfully reproduced the linear policy (mimic loss ≈ 0.0002). These weights served as warm start for `fmincon` fine-tuning.

Result: Robot fell at step 14 (time 0.042 s). Final state: $[2.17, 38.98, -1.56, -32.16]$.

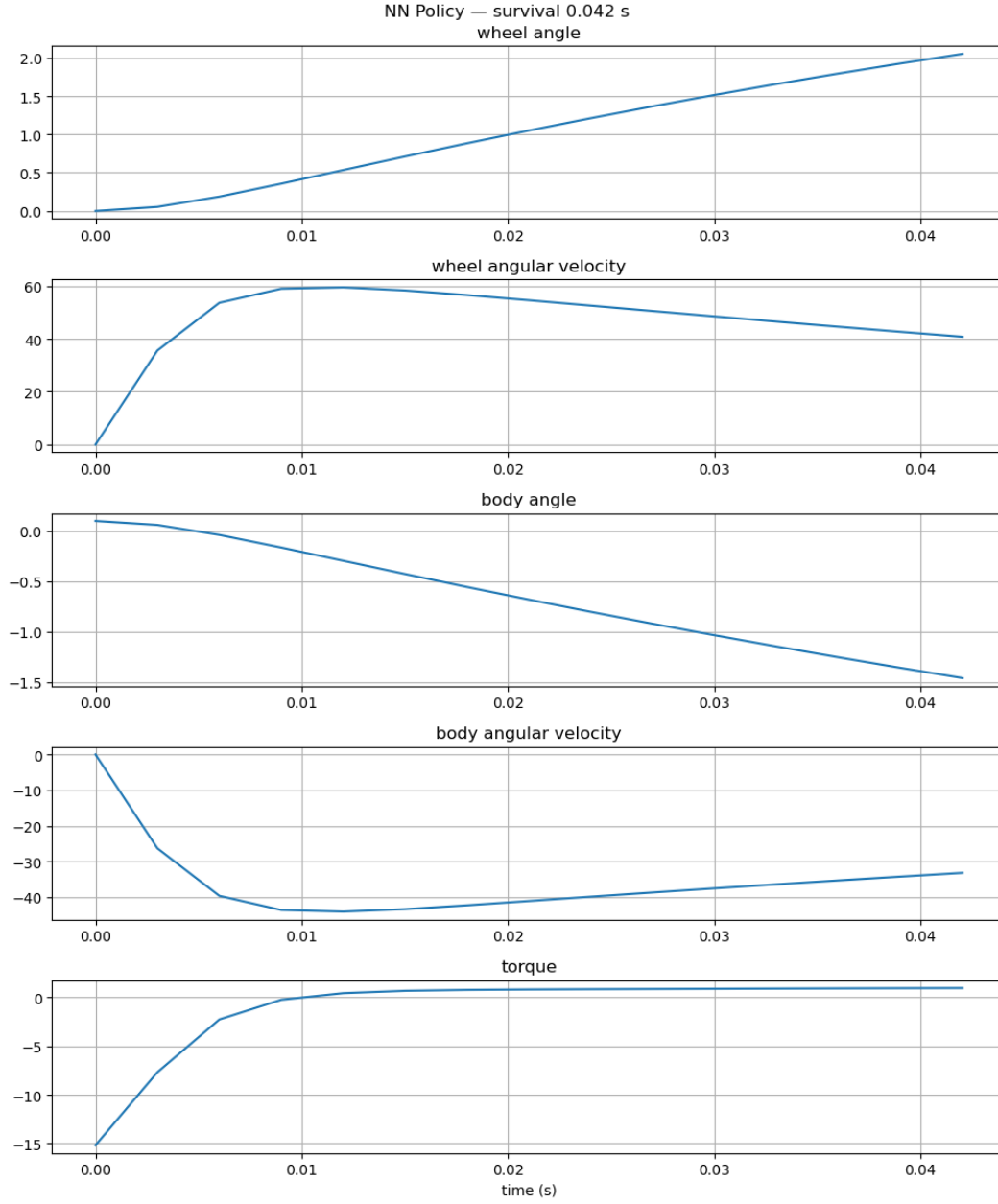


Figure 3: Neural network policy from $x_0 = [0, 0, 0.1, 0]$. Fell at 0.042 s not better than the linear controller.

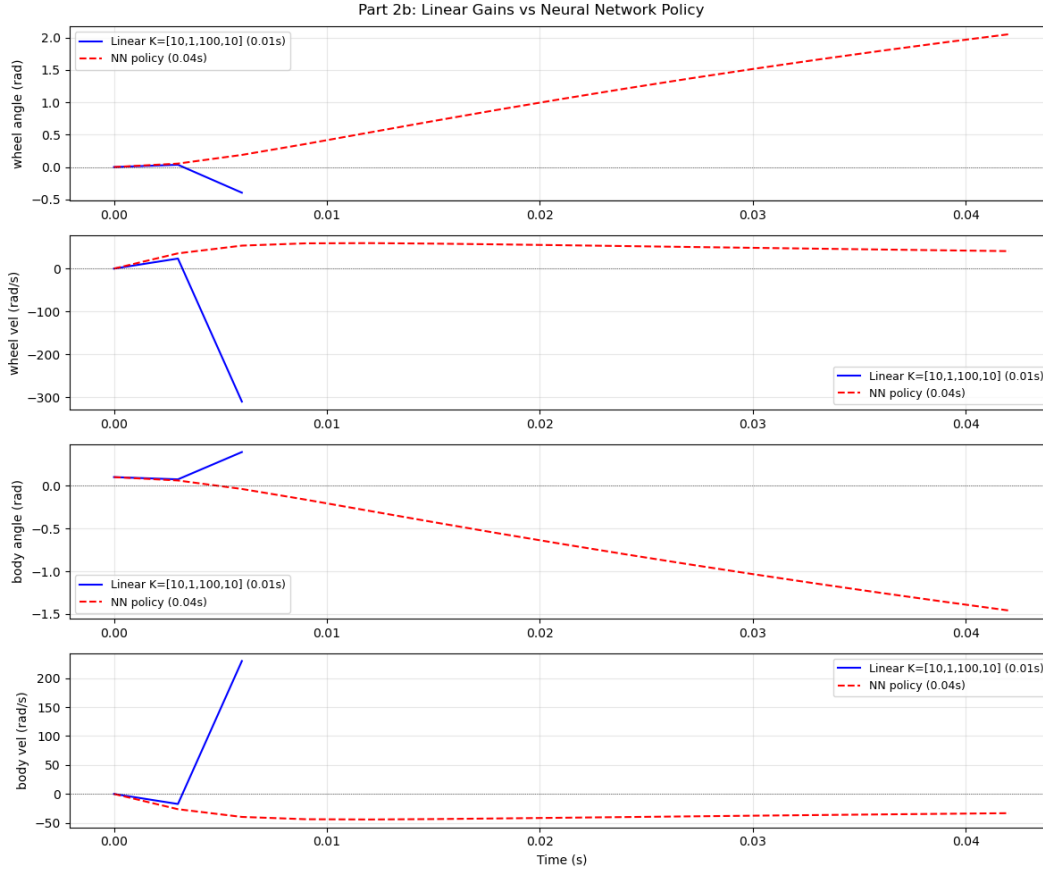


Figure 4: Comparison: linear gains (blue, falls at 0.006 s) vs NN policy (red dashed, falls at 0.042 s). The NN survives slightly longer but neither controller stabilizes the robot.

4.4 What Changes with a Neural Network?

- **More parameters:** 4 gains vs 13 weights. Optimization becomes much harder.
- **Different methods needed:** `fminunc` works for 4 parameters but fails for 13+. Imitation learning was necessary to produce a usable starting point.
- **More expressive, harder to train:** The NN could in principle represent nonlinear policies that outperform linear ones. But actually finding those weights is the hard part the NN only reproduced the linear controller and did not improve on it.
- **No guaranteed improvement:** Imitation learning produces a controller that matches the teacher but cannot exceed it without a better training objective (e.g., reinforcement learning).

5 What I Learned

- Manual tuning is a waste of time for more than 2 parameters.
- LQR gives gains that work locally but fail for large disturbances valid only near the linearization point.

- Direct optimization with `fminunc` / `fmincon` works reasonably for 4 parameters but gets slow for higher-dimensional problems.
- For neural networks, direct optimization with standard optimizers is hopeless. The search space is too large and the cost landscape too noisy.
- Imitation learning (cloning a working controller) is the easiest way to get a neural network policy that works, but it will not outperform the original controller without a better training signal.
- When optimizing learned functions: more data helps, constraints help, but hallucination is a fundamental risk that cannot be eliminated without evaluating the true function at the found optimum.